

Contents

List of Figures	vi
1 DEVELOPMENT ENVIRONMENT SETUP FOR JAVA WEB APPLICATIONS	1
1.1 Objectives	1
1.2 Introduction	1
1.3 Software Requirements	2
1.4 Task 1: Install Java Development Kit (JDK)	2
1.4.1 Step 1: Download JDK	2
1.4.2 Step 2: Install JDK	2
1.4.3 Step 3: Verify JDK Installation	2
1.5 Task 2: Install XAMPP	3
1.5.1 Step 1: Download XAMPP	3
1.5.2 Step 2: Install XAMPP without Tomcat	4
1.5.3 Step 3: Verify XAMPP Installation	9
1.6 Task 3: Change the Default Root Password of MySQL	11
1.6.1 Step 1: Start Apache and MySQL Services	11
1.6.2 Step 2: Run Command	12
1.7 Task 4: Tomcat 9 Installation	18
1.7.1 Step 1: Download Apache Tomcat 9	18
1.7.2 Step 2: Extract Apache Tomcat 9	19
1.7.3 Step 3: Start Tomcat Server	19
1.7.4 Step 4: Verify Installation	20
1.8 Task 5: NetBeans IDE Installation	21
1.8.1 Step 1: Download Apache NetBeans 29	21
1.8.2 Step 2: Extract and Install NetBeans 29	21
1.8.3 Step 3: Configure NetBeans 29	23
1.8.4 Step 5: Configure Tomcat in NetBeans	23
1.9 Task 6: Writing a Simple Java Servlet	26
1.9.1 Step 1: Create New Project	26
1.9.2 Step 2: Create New Servlet	29
1.9.3 Step 3: Run Servlet	32
1.10 Task 7: Writing a Simple JSP Program	34
1.10.1 Step 1: Create a New Project	34
1.10.2 Step 2: Create a New JSP File	36
1.10.3 Step 3: Run JSP File	39
1.11 Task 8: Use Java Reference Datatype/Class Wrapper in JSP	41
1.11.1 Step 1: Create a New JSP File	41

1.12	Exercise	43
2	SERVLET: DATA SHARING AND DATABASE MANAGEMENT	45
2.1	Objectives	45
2.2	Introduction	46
2.3	Software Requirements	46
2.4	Task 1: Data Sharing in Servlet	46
2.4.1	Step 1: Project Setup	46
2.4.2	Step 2: Create HTML Login Page	48
2.4.3	Step 3: Create a Login Servlet	48
2.4.4	Step 4: Create an Account Servlet	51
2.4.5	Step 5: Run the Project	52
2.5	Task 2: Creating A Table in MySQL Database	53
2.5.1	Step 1: Start Apache and MySQL	53
2.5.2	Step 2: Create a New Database	53
2.5.3	Step 3: Create a New MySQL Table	53
2.6	Task 3: Setting the Environment of Web Application for Database Connection	54
2.6.1	Step 1: Create a New Web Application Project	54
2.6.2	Step 2: Prepare the MySQL Connector	55
2.7	Task 4: Using Servlets for Database CRUD Operations	57
2.7.1	Step 1: Create HTML Form for Data Entry	57
2.7.2	Step 2: Create the Insert Servlet (Create)	58
2.7.3	Step 3: Create the View Servlet (Read)	59
2.7.4	Step 4: Create the Delete Servlet (Delete)	62
2.7.5	Step 5: Create the Update Servlet (Update)	63
2.8	Task 5: Using Servlets, DAO and Java Beans for Database CRUD Operations	65
2.8.1	Step 1: Copy and Rename the Project	66
2.8.2	Step 2: Create the User JavaBean (Model)	66
2.8.3	Step 3: Create the Data Access Object (UserDAO)	68
2.8.4	Step 4: Modify the Existing Servlets (Controller)	72
2.9	Exercise	77
3	JSP: SCRIPTLET, PAGE DIRECTIVE & INCLUDE DIRECTIVE	79
3.1	Objectives	79
3.2	Introduction	79
3.3	Software Requirements	80
3.4	Task 1: Passing Data from Main JSP's Page to Other JSP's Page	80
3.4.1	Step 1: Create A New JSP Page	80
3.4.2	Step 2: Write an HTML Markup	81
3.5	Task 2: Using Mathematics Operations in JSP	84
3.5.1	Step 1: Create A New JSP Page	85
3.5.2	Step 2: Write an HTML Markup	85
3.5.3	Step 3: Run the <code>Calculator.jsp</code>	86
3.6	Task 3: Populate an Array Values into HTML's Table	89
3.6.1	Step 1: Create A New JSP Page	89
3.6.2	Step 2: Write An HTML Markup	89
3.6.3	Step 3: Run the <code>populateArray.jsp</code> File	90

3.7	Task 4: Perform Calculation of Car Loan	90
3.7.1	Step 1: Create A New HTML File	90
3.7.2	Step 2: Create An HTML Markup	91
3.7.3	Step 3: Create <code>processCalculateCarLoan.jsp</code>	92
3.8	Task 5: Using JSP Page Directive to Call Java API	93
3.9	Task 6: Use JSP Include directive for JSP Page	98
4	JSP: SCRIPTLET, EXPRESSION & STANDARD ACTIONS	103
4.1	Objectives	103
4.2	Introduction	103
4.3	Software Requirements	104
4.4	Task 1: Using JSP Scripting	110
4.4.1	Step 1: Create <code>customer.html</code>	110
4.4.2	Step 2: Create <code>processCustomer.jsp</code>	111
4.4.3	Step 3: Compile and Run the Application	113
4.5	Task 2: Using JSP (Scripting, Declaration and Expression)	115
4.5.1	Step 1: Create <code>currencyConversion.html</code>	115
4.5.2	Step 2: Create <code>processCurrency.jsp</code>	116
4.5.3	Step 3: Compile and Run the Application	118
4.6	Task 3: Using JSP Standard Action (Include and Param)	120
4.6.1	Step 1: Create <code>jspParameter.jsp</code>	120
4.6.2	Step 2: Create <code>subjectInfo.jsp</code>	122
4.6.3	Step 3: Compile and Run the Application	123
4.7	Task 4: Using JSP Standard Action (Forward)	124
4.7.1	Step 1: Create <code>forward.jsp</code>	124
4.7.2	Step 2: Create <code>forwardInfo.jsp</code>	125
4.7.3	Step 3: Compile and Run the Application	126
4.8	Task 5: Using JSP Scriptlet to Construct Business Logic	127
4.8.1	Step 1: Create Project and JSP File <code>insuranceQuotation.jsp</code>	127
4.8.2	Step 2: Design Insurance Quotation Interface	127
4.8.3	Step 3: Create <code>processInsuranceQuo.jsp</code>	128
4.8.4	Step 4: Implement Business Logic Using JSP Scriptlet	129
4.8.5	Step 5: Compile and Run the Application	130
5	JSP: JAVABEANS AND STANDARD TAG LIBRARY (JSTL)	135
5.1	Objectives	135
5.2	Introduction	136
5.3	Software Requirements	136
5.4	Task 1: Using Scriptlet to Access a Simple JavaBeans	136
5.4.1	Step 1: Create the JavaBean	136
5.4.2	Step 2: Access Bean via Scriptlet	137
5.5	Task 2: Problem Solving using JavaBeans	138
5.5.1	Step 1: Create a Calculator Bean	138
5.5.2	Step 2: Create Form and Process Page	139
5.6	Task 3: Installing JSTL Taglibs	140
5.6.1	Step 1: Download and Import JSTL Libraries	141
5.6.2	Step 2: Test the Installation and Formatting Tags	142
5.7	Task 4: Using Java Standard Tag Library (JSTL) Core Tags	144

5.7.1	Step 1: Variables and Conditionals	144
5.8	Task 5: Using JSP Standard Tag Library (JSTL) for Collections	145
5.8.1	Step 1: Create the Servlet (Controller)	145
5.8.2	Step 2: Create the View (JSTL forEach)	146
5.9	Task 6: Database Access using JSTL SQL Tags	147
5.9.1	Step 1: Create Database and Table	148
5.9.2	Step 2: Prepare the Database Driver	148
5.9.3	Step 3: Create the JSTL SQL Page	149
5.10	Exercise: Employee Payroll Display System	151
6	JSP: SAVING AND RETRIEVING DATA FROM DATABASE	153
6.1	Objectives	153
6.2	Introduction	153
6.3	Software Requirements	154
6.4	Task 1: Using JSP Page to Access a Simple MySQL Database	155
6.4.1	Step 1 - Create a table namely <code>FirstTable</code> using phpMyAdmin	155
6.4.2	Step 2 - Create <code>SampleInsertionRecord.jsp</code> to insert data in <code>FirstTable</code> table.	156
6.4.3	Step 3 - go to the database.	158
6.5	Task 2: Create Records via JSP Page	160
6.6	Task 3: Create Records Constrained by Regular Expression in JSP	166
6.6.1	Step 1 - Create a table <code>student</code> using phpMyAdmin.	166
6.6.2	Step 2 - Create <code>insertStudent.jsp</code> as a main interface to register a new student.	167
6.6.3	Step 3 - Create <code>Student</code> JavaBeans	167
6.6.4	Step 4 - Create <code>processStudent.jsp</code> to insert record into the database	169
6.6.5	Step 5 - Create <code>errorStudent.jsp</code> to display any error message	171
6.6.6	Step 6 - Running the program and create a new database	171
6.7	Task 4: Perform Retrieving Records Via JSP Page	173
6.8	Task 5: Create A Record Using JSP Model 1	176
7	SESSION MANAGEMENT, FILE UPLOAD & MVC	185
7.1	Objectives	185
7.2	Introduction	185
7.3	Database Preparation	186
7.4	Task 1: Building the Model (JavaBean & DAO)	186
7.4.1	Step 1: Create the <code>StudentBean</code>	187
7.4.2	Step 2: Create the <code>StudentDAO</code>	187
7.5	Task 2: The Unified Controller (UserServlet)	190
7.5.1	Step 1: Create UserServlet with web.xml Mapping	190
7.6	Task 3: The Views and Session Security	192
7.6.1	Step 1: Registration Form	193
7.6.2	Step 2: Login Form	193
7.6.3	Step 3: Secure Dashboard JSP	194
7.7	Task 4: Debugging and Viewing Server Logs	196
7.7.1	Step 1: Simulate a Backend Error	196
7.7.2	Step 2: Inspecting the Apache Tomcat Output Window	196

7.7.3	Step 3: Viewing the Full Server Log via Services Tab	197
7.8	Exercise: Student Subject Management System (CRUD)	199
	APPENDIX: Lecturer Solution Key (Subject CRUD)	201
8	AN MVC EXAMPLE WITH SERVLETS AND JSP	205
8.1	Objectives	205
8.2	Introduction	205
8.3	Software Requirements	206

List of Figures

1.1	XAMPP Download Platform	3
1.2	XAMPP Setup Wizard Startup Screen	5
1.3	XAMPP Select Components	5
1.4	XAMPP Installation Directory	6
1.5	XAMPP Language	7
1.6	XAMPP Installation Ready	7
1.7	XAMPP Installation Progress	8
1.8	XAMPP Installation Finish	8
1.9	XAMPP Control Panel	9
1.10	XAMPP Control Panel Running	10
1.11	XAMPP Dashboard	10
1.12	phpMyAdmin Interface	11
1.13	XAMPP Control Panel-Click Shell	11
1.14	Verify Password	12
1.15	Error Page	13
1.16	Apache Tomcat9 Download Page	18
1.17	Start Tomcat Server	19
1.18	Apache Tomcat Homepage	20
1.19	Apache NetBeans 29 Download Page	21
1.20	Extract netbeans-29-bin.zip	22
1.21	Netbean Executable File	22
1.22	Netbean IDE GUI	23
1.23	Configure Server	24
1.24	Add Server	24
1.25	Add Server	25
1.26	Add Server	25
1.27	Add Server	26
1.28	Create New Project	27
1.29	Create New Web Application	27
1.30	Enter Project Details	28
1.31	Create New Servlet	29
1.32	Name the Servlet	30
1.33	Configure Servlet Deployment	30
1.34	HelloServlet.java File	31
1.35	<code>processRequest()</code> Method	31
1.36	Modified <code>processRequest()</code> Method	31
1.37	Run the Servlet	32

1.38	Dialog Box Call to /HelloServlet	32
1.39	Output	33
1.40	Modified <code>doGet()</code> Method	33
1.41	Dialog Box with Passing Parameter	34
1.42	Output	34
1.43	Create New Web Application	35
1.44	Enter Project Details	36
1.45	Configure the Server	36
1.46	Create a New JSP File	37
1.47	Enter Name and Location	37
1.48	Modified JSP File	38
1.49	Compile JSP File	38
1.50	Compile JSP File	38
1.51	Run JSP File	39
1.52	Output	40
1.53	Start/Stop Apache Tomcat	40
1.54	Create <code>useJavaObject.jsp</code>	41
1.55	<code>useJavaObject.jsp</code>	42
1.56	Output	43
2.1	Create New Servlet	49
2.2	Create Login Servlet	49
2.3	Configure servlet deployment	50
2.4	Creating the “CRUDusingServlet” Project	55
2.5	Downloading MySQL Connector/J	55
2.6	Extracting the MySQL Connector/J Archive	56
2.7	Adding External Library to Project	56
2.8	Importing MySQL Connector/J Library	57
3.1	Create new JSP	81
3.2	New JSP	81
3.3	<code>memberRegister.jsp</code> Interface	82
3.4	Filling input <code>memberRegister.jsp</code>	84
3.5	Output in <code>memberProcessing.jsp</code>	84
3.6	Create new JSP	85
3.7	New JSP	85
3.8	<code>calculator.jsp</code> Interface	86
3.9	Key-in Filename	89
3.10	Output from Array	90
3.11	Creating New HTML File	91
3.12	Interface Output for Car Loan Calculation HTML	91
3.13	Input for Car Loan Calculation	92
3.14	Output for Car Loan Calculation	93
3.15	New JSP	93
3.16	Key-in Filename	94
3.17	Save All	96
3.18	Run	96
3.19	Success Message	96

3.20	ArrayList Output	97
3.21	Tomcat Before Run	97
3.22	Tomcat After Run	97
3.23	Tomcat Success Message	98
3.24	mainPage.jsp	98
3.25	headerPage.jsp Output	98
3.26	footerPage.jsp Output	99
3.27	mainPage.jsp Output	99
4.1	Creating customer.html	111
4.2	Interface of customer.html	111
4.3	Creating processCustomer.jsp	112
4.4	Example input in customer.html	114
4.5	Output after processing customer input	114
4.6	Creating currencyConversion.html	115
4.7	Interface of currencyConversion.html	116
4.8	Creating processCurrency.jsp	116
4.9	Interface of currencyConversion.html	119
4.10	Input example in currencyConversion.html	119
4.11	Processing result of currency conversion	120
4.12	Creating jspParameter.jsp	121
4.13	Creating subjectInfo.jsp	122
4.14	Output of jspParameter.jsp	123
4.15	Creating forward.jsp	124
4.16	Creating forwardInfo.jsp	125
4.17	Output of forwarded information	126
4.18	Creating insuranceQuotation.jsp	127
4.19	Insurance Quotation Interface	128
4.20	Creating processInsuranceQuo.jsp	128
4.21	Insurance Quotation Output	131
5.1	Download jstl-1.2.jar	141
6.1	Data Insertion Record	158
6.2	Data in MySQL	159
6.3	insertAuthor Interface	161
6.4	Insert Author Succeed	165
6.5	insertStudent Interface	167
6.6	Insert Student Succeed	172
6.7	Student Query	176
6.8	Marathon Registration GUI	177
6.9	Marathon Registration Succeed	182
7.1	Viewing the Server Log in NetBeans	197
8.1	Project Schema	222
8.2	Index	223
8.3	Employee List	223
8.4	Add Employee	223

8.5	Edit Employee	224
8.6	Delete Employee	224

Lab Module 8

AN MVC EXAMPLE WITH SERVLETS AND JSP

8.1 Objectives

The primary goal of this lab is to teach students how to build a structured web application by separating concerns using the Model-View-Controller (MVC) design pattern. Upon completion, students should be able to:

1. **Understand MVC Components:** Define and implement the roles of the **Model** (data/logic), **View** (visualization), and **Controller** (flow control).
2. **Implement CRUD Functionality:** Create a complete web application capable of performing **Create**, **Read**, **Update**, and **Delete** operations on a MySQL database.
3. **Apply Data Access Object (DAO) Patterns:** Use a helper class (DAO) to manage database connections and SQL execution, keeping business logic clean.
4. **Utilize Servlet Controllers:** Develop a centralized Servlet to handle URL mapping and direct data flow between the Model and the View.
5. **Design Dynamic Views:** Create JSP pages that use JSTL and Expression Language (EL) to display data without heavy Java scriptlets.

8.2 Introduction

The Model-View-Controller (MVC) pattern is an industry-standard architectural design used to separate an application's concerns, making it easier to maintain and scale. In a JSP and Servlet environment, MVC minimizes the mixing of HTML and Java code.

In this lab, students develop an **Employee Directory** application. The **Controller** (Servlet) manages requests from the web browser, invokes the **Model** (Java Classes and DAO) to interact with the **MySQL Database**, and finally forwards the results to the **View** (JSP Pages) for client-side visualization. This separation ensures that the presentation layer remains independent of the underlying business logic.

8.3 Software Requirements

To successfully build and run the Employee Management application, the following software environment is required:

- **Web & Database Server Bundle: XAMPP:** to provide the Apache web server and MySQL database.
- **Database Management Tool: phpMyAdmin:** for schema and table creation
- **Integrated Development Environment (IDE):** NetBeans.
- **Database Driver:** MySQL Java Connector.
- **Web Browser:** Any modern browser (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge) for testing and viewing output.

8.3 Steps Involved in the Lab

1. Capture the employee records and store it into the database.
2. Fetch the employee records from the database and display it on the JSP.
3. Update the existing employee records into the database.
4. Delete the employee records from the database.

1. Create table employees in COMPANY database schema.

```
CREATE DATABASE IF NOT EXISTS Company; Use Company;
```

```
CREATE TABLE IF NOT EXISTS employees (
    id INT NOT NULL AUTO_INCREMENT,
    Name VARCHAR(60),
    Email VARCHAR(50),
    Position VARCHAR(15),
    PRIMARY KEY (id)
);
```

2. Create a new web application project, named Employee_Management.
3. Create three Java class that representing :
 - EmployeeDAO.java (act as a Data Access Object (DAO) and to open /close database connection) - name the package as com.DAO.

```
package com.DAO;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import com.Model.Employee;

public class EmployeeDAO {
    Connection connection = null;
    private String jdbcURL = "jdbc:mysql://localhost:3306/company";
    private String jdbcUsername = "root";
    private String jdbcPassword = "admin";

    private static final String INSERT_EMPLOYEES_SQL = "INSERT INTO
employees (name, email, position) VALUES " +
        " (?, ?, ?);";

    private static final String SELECT_EMPLOYEE_BY_ID = "select id,
name,email,position from employees where id =?";
    private static final String SELECT_ALL_EMPLOYEES = "select * from
employees";
```

```

private static final String DELETE_EMPLOYEES_SQL = "delete from
employees where id = ?";
private static final String UPDATE_EMPLOYEES_SQL = "update
employees set name = ?,email= ?, position =? where id = ?";

public EmployeeDAO() {}

protected Connection getConnection() {
    Connection connection = null;
    try {
        Class.forName("com.mysql.jdbc.Driver");
        connection = DriverManager.getConnection(jdbcURL,
            jdbcUsername, jdbcPassword);
    } catch (SQLException e) {
        e.printStackTrace();
    } catch (ClassNotFoundException e) {
        e.printStackTrace();
    }
    return connection;
}

public void insertEmployee(Employee employee) throws SQLException {
    System.out.println(INSERT_EMPLOYEES_SQL);
    // try-with-resource statement will auto close the connection.
    try (Connection connection = getConnection();
        PreparedStatement preparedStatement = connection.
            prepareStatement(INSERT_EMPLOYEES_SQL)) {
        preparedStatement.setString(1, employee.getName());
        preparedStatement.setString(2, employee.getEmail());
        preparedStatement.setString(3, employee.getPosition());
        System.out.println(preparedStatement);
        preparedStatement.executeUpdate();
    } catch (SQLException e) {
        printSQLException(e);
    }
}

public Employee selectEmployee(int id) {
    Employee employee = null;
    // Step 1: Establishing a Connection
    try (Connection connection = getConnection());

```

```

        // Step 2: Create a statement using connection object
        PreparedStatement preparedStatement = connection.
        prepareStatement(SELECT_EMPLOYEE_BY_ID);) {
        preparedStatement.setInt(1, id);

        System.out.println(preparedStatement);
        // Step 3: Execute the query or update query
        ResultSet rs = preparedStatement.executeQuery();

        // Step 4: Process the ResultSet object.
        while (rs.next()) {
            String name = rs.getString("name");
            String email = rs.getString("email");
            String position = rs.getString("position");
            employee = new Employee(id, name, email, position);
        }
    } catch (SQLException e) {
        printSQLException(e);
    }
    return employee;
}

public List < Employee > selectAllEmployees() {
    // using try-with-resources to avoid closing resources (boiler
    plate code)
    List < Employee > employees = new ArrayList < > ();
    // Step 1: Establishing a Connection
    try (Connection connection = getConnection();
        // Step 2: Create a statement using connection object
        PreparedStatement preparedStatement = connection.
        prepareStatement(SELECT_ALL_EMPLOYEES);) {
        System.out.println(preparedStatement);
        // Step 3: Execute the query or update query
        ResultSet rs = preparedStatement.executeQuery();

        // Step 4: Process the ResultSet object.
        while (rs.next()) {
            int id = rs.getInt("id");
            String name = rs.getString("name");
            String email = rs.getString("email");
            String position = rs.getString("position");

```



```

        employees.add(new Employee(id, name, email, position));
    }
} catch (SQLException e) {
    printSQLException(e);
}
return employees;
}

public boolean deleteEmployee(int id) throws SQLException {
    boolean rowDeleted;
    try (Connection connection = getConnection();
        PreparedStatement statement = connection.
            prepareStatement(DELETE_EMPLOYEES_SQL);) {
        statement.setInt(1, id);
        rowDeleted = statement.executeUpdate() > 0;
    }
    return rowDeleted;
}

public boolean updateEmployee(Employee employee) throws
SQLException {
    boolean rowUpdated;
    try (Connection connection = getConnection();
        PreparedStatement statement = connection.
            prepareStatement(UPDATE_EMPLOYEES_SQL);) {
        statement.setString(1, employee.getName());
        statement.setString(2, employee.getEmail());
        statement.setString(3, employee.getPosition());
        statement.setInt(4, employee.getId());

        rowUpdated = statement.executeUpdate() > 0;
    }
    return rowUpdated;
}

private void printSQLException(SQLException ex) {
    for (Throwable e: ex) {
        if (e instanceof SQLException) {
            e.printStackTrace(System.err);
            System.err.println("SQLState: " + ((SQLException)
                e).getSQLState());

```

```

        System.err.println("Error Code: " + ((SQLException)
e).getErrorCode());
        System.err.println("Message: " + e.getMessage());
        Throwable t = ex.getCause();
        while (t != null) {
            System.out.println("Cause: " + t);
            t = t.getCause();
        }
    }
}
}
}
}

```

- Employee.java (act as a JavaBeans to represent business object), and

```

package com.Model;

public class Employee {
    protected int id;
    protected String name;
    protected String email;
    protected String position;

    public Employee() {}

    public Employee(String name, String email, String position) {
        super();
        this.name = name;
        this.email = email;
        this.position = position;
    }

    public Employee(int id, String name, String email, String
position) {
        super();
        this.id = id;
        this.name = name;
        this.email = email;
        this.position = position;
    }
}

```

```

public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getPosition() {
    return position;
}

public void setPosition(String position) {
    this.position = position;
}
}

```

- EmployeeServlet.java (act to perform CRUD process)

```

package com.WEB;

import java.io.IOException;
import java.sql.SQLException;
import java.util.List;

```

```

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.DAO.EmployeeDAO;
import com.Model.Employee;

@WebServlet("/")
public class EmployeeServlet extends HttpServlet {
    private EmployeeDAO employeeDAO;

    public void init() {
        employeeDAO = new EmployeeDAO();
    }

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        String action = request.getServletPath();

        try {
            switch (action) {
                case "/new":
                    showNewForm(request, response);
                    break;
                case "/insert":
                    insertEmployee(request, response);
                    break;
                case "/delete":
                    deleteEmployee(request, response);
                    break;
                case "/edit":

```

```

        showEditForm(request, response);
        break;
    case "/update":
        updateEmployee(request, response);
        break;
    default:
        listEmployee(request, response);
        break;
    }
} catch (SQLException ex) {
    throw new ServletException(ex);
}
}

private void listEmployee(HttpServletRequest request,
    HttpServletResponse response)
    throws SQLException, IOException, ServletException {
    List < Employee > listEmployee =
        employeeDAO.selectAllEmployees();
    request.setAttribute("listEmployee", listEmployee);
    RequestDispatcher dispatcher =
        request.getRequestDispatcher("employeeList.jsp");
    dispatcher.forward(request, response);
}

private void showNewForm(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {
    RequestDispatcher dispatcher =
        request.getRequestDispatcher("employeeForm.jsp");
    dispatcher.forward(request, response);
}

private void showEditForm(HttpServletRequest request,
    HttpServletResponse response)
    throws SQLException, ServletException, IOException {
    int id = Integer.parseInt(request.getParameter("id"));
    Employee existingEmployee = employeeDAO.selectEmployee(id);
    RequestDispatcher dispatcher =
        request.getRequestDispatcher("employeeForm.jsp");
    request.setAttribute("employee", existingEmployee);

```

```

        dispatcher.forward(request, response);
    }

    private void insertEmployee(HttpServletRequest request,
        HttpServletResponse response)
        throws SQLException, IOException {
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String position = request.getParameter("position");
        Employee newEmployee = new Employee(name, email, position);
        employeeDAO.insertEmployee(newEmployee);
        response.sendRedirect("list");
    }

    private void updateEmployee(HttpServletRequest request,
        HttpServletResponse response)
        throws SQLException, IOException {
        int id = Integer.parseInt(request.getParameter("id"));
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String position = request.getParameter("position");

        Employee employee = new Employee(id, name, email, position);
        employeeDAO.updateEmployee(employee);
        response.sendRedirect("list");
    }

    private void deleteEmployee(HttpServletRequest request,
        HttpServletResponse response)
        throws SQLException, IOException {
        int id = Integer.parseInt(request.getParameter("id"));
        employeeDAO.deleteEmployee(id);
        response.sendRedirect("list");
    }
}

```

4. Create these files:

- File web.xml

Java web applications use a deployment descriptor file to determine how URLs

map to servlets, which URLs require authentication, and other information. This file is named web.xml, and resides in the app's WAR under the WEB-INF/ directory. web.xml is part of the servlet standard for web applications.

```
<!-- Ensure CSS, JavaScript and image files are served correctly
      when using servlet mapping "/" -->

<servlet-mapping>
    <servlet-name>default</servlet-name>
    <url-pattern>*.css</url-pattern>
</servlet-mapping>

<servlet-mapping>
    <servlet-name>default</servlet-name>
    <url-pattern>*.js</url-pattern>
</servlet-mapping>

<servlet-mapping>
    <servlet-name>default</servlet-name>
    <url-pattern>*.png</url-pattern>
</servlet-mapping>
```

Important: When a servlet is mapped using `<url-pattern>/</url-pattern>`, all incoming requests are routed through that servlet, including requests for static resources such as CSS, JavaScript, and image files. To ensure these resources are served correctly by the application server, explicit mappings to the default servlet should be added in web.xml as shown above. Failure to do so may result in missing stylesheets, JavaScript files, or images when deploying larger web applications such as Final Year Project (PITA) systems.

- EmployeeForm.jsp (used for Add and Edit/Update process)

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html>
<html>
    <head>
        <title>Employee Management Application</title>
        <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/
bootstrap/4.3.1/css/bootstrap.min.css"
```

```

        integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/
        iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-md navbar-dark"
            style="background-color: tomato">
            <div>
                <a href="" class="navbar-brand"> Employee Management
                App </a>
            </div>

            <ul class="navbar-nav">
                <li><a href="<%=request.getContextPath()%>/list"
                    class="nav-link">Employees</a></li>
            </ul>
        </nav>
    </header>
    <br>
    <div class="container col-md-5">
        <div class="card">
            <div class="card-body">
                <c:if test="${employee != null}">
                    <form action="update" method="post">
                </c:if>
                <c:if test="${employee == null}">
                    <form action="insert" method="post">
                </c:if>

                <h2>
                    <c:if test="${employee != null}">
                        Edit Employee
                    </c:if>
                    <c:if test="${employee == null}">
                        Add New Employee
                    </c:if>
                </h2>

                <c:if test="${employee != null}">
                    <input type="hidden" name="id" value="
                    <c:out value='${employee.id}' />" />

```



```

</c:if>

<fieldset class="form-group">
  <label>Employee Name</label>
  <input type="text" value="<c:out
value='${employee.name}' />"
        class="form-control" name="name"
        required="required">
</fieldset>

<fieldset class="form-group">
  <label>Employee Email</label>
  <input type="text" value="<c:out
value='${employee.email}' />"
        class="form-control" name="email">
</fieldset>

<fieldset class="form-group">
  <label>Employee Position</label>
  <input list="positionList" id="position"
        class="form-control"
        name="position" value="<c:out
value='${employee.position}' />">
  <datalist id="positionList">
    <option value="Manager">
    <option value="Head of Dept">
    <option value="Supervisor">
    <option value="Director">
  </datalist>
</fieldset>

  <button type="submit" class="btn btn-
success">Save</button>
</form>
</div>
</div>
</div>
</body>
</html>

```

- EmployeeList.jsp (used for displaying all employee records)

```

<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html>
<html>
<head>
    <title>Employee Management Application</title>
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/
bootstrap/4.3.1/css/bootstrap.min.css"
        integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/
iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-md navbar-dark"
style="background-color: tomato">
            <div>
                <a href="" class="navbar-brand"> Employee
Management App </a>
            </div>
            <ul class="navbar-nav">
                <li><a href="<%=request.getContextPath()%>/list"
class="nav-link">Employees</a></li>
            </ul>
        </nav>
    </header>
    <br>
    <div class="row">
        <div class="container">
            <h3 class="text-center">List of Employees</h3>
            <hr>
            <div class="container text-left">
                <a href="<%=request.getContextPath()%>/new"
class="btn btn-success">Add New Employee</a>
            </div>
            <br>
            <table class="table table-bordered">
                <thead>
                    <tr>
                        <th>ID</th>

```



```

<head>
    <title>Error page</title>
</head>
<body>
    <center>
        <h1>Error</h1>
        <h2><%=exception.getMessage() %><br/> </h2>
    </center>
</body>
</html>

```

- index.jsp

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <title>User Management Application</title>
        <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css"
            integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">
    </head>
    <body>
        <h1>Application MVC system for Employee Management</h1><br>

        <ul>
            <li><a href="http://localhost:8080/Lab8/list"> All Employee List </a></li>
            <li><a href="http://localhost:8080/Lab8/new"> Add a New Employee </a></li>
            <li><a href="http://localhost:8080/Lab8/list"> Edit Employee </a></li>
        </ul>

    </body>
</html>

```

5. Add these libraries:

- JSTL Library

- MySQL Java connector

6. The project schema should be like this:

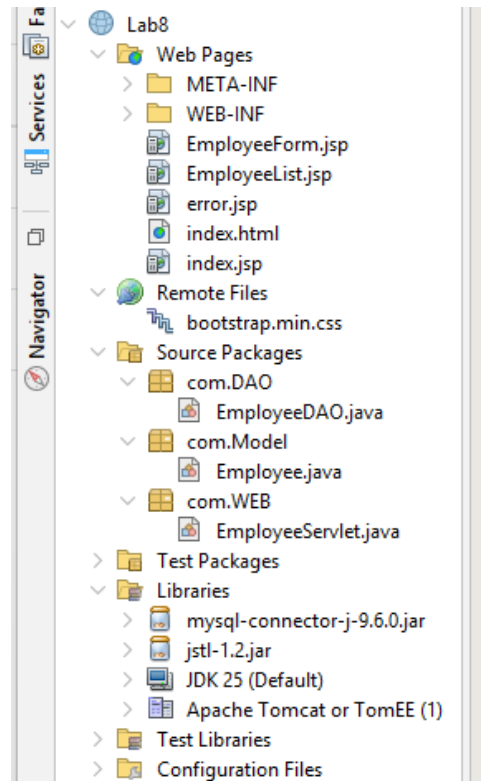


Figure 8.1: Project Schema

7. Running the program and try CRUD process:

(a) Run index.jsp page.

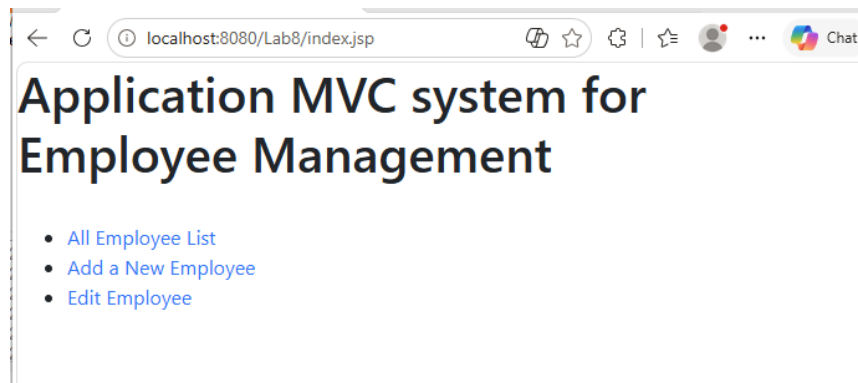


Figure 8.2: Index

(b) Click List All User button to show all records.

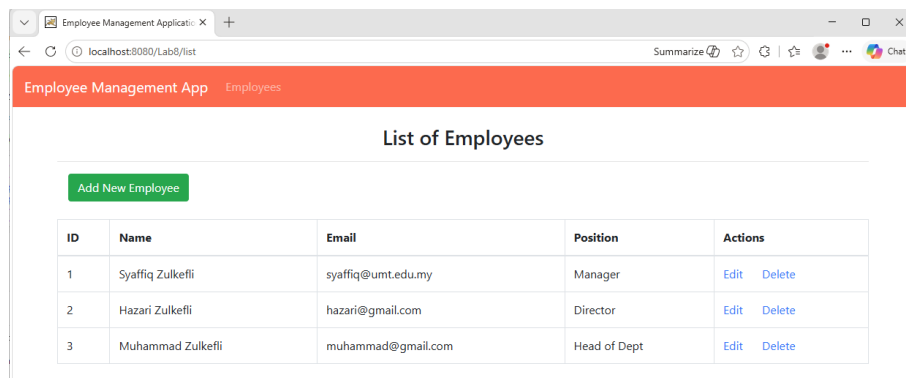


Figure 8.3: Employee List

(c) Click Add User button to create new record.

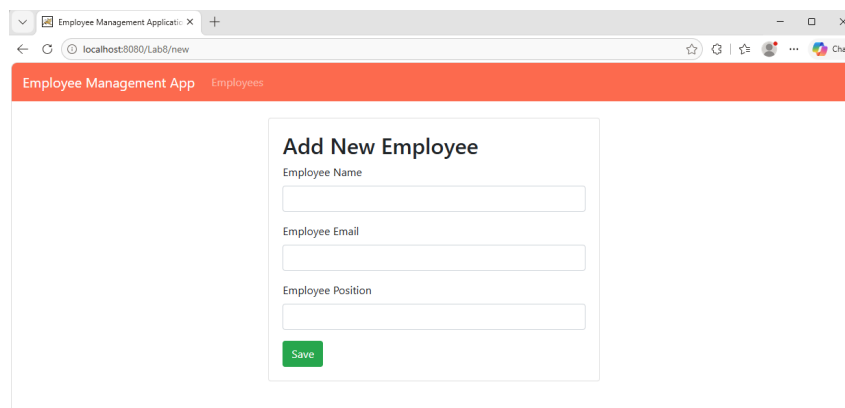


Figure 8.4: Add Employee

(d) Click hyperlink Update to do edit/update an existing record.

Employee Management App Employees

Edit Employee

Employee Name
Hazari Zulkefli

Employee Email
hazari@gmail.com

Employee Position
Director

Save

Figure 8.5: Edit Employee

(e) Click hyperlink Delete to do delete an existing record.

Employee Management App Employees

List of Employees

Add New Employee

ID	Name	Email	Position	Actions
1	Syaffiq Zulkefli	syaffiq@umt.edu.my	Manager	Edit Delete
2	Hazari Zulkefli	hazari@gmail.com	Director	Edit Delete

Figure 8.6: Delete Employee

Exercise

Implement MVC for Car Shop

1. Using this database schema, create an MVC Application [CRUD] for Car Shop, using JSP, Servlet, and MySQL.

```
CREATE DATABASE if not EXISTS carshop;
USE carshop;

CREATE TABLE if not EXISTS CarPricelist(
    Car_id INT NOT NULL AUTO_INCREMENT,
    Brand VARCHAR(15),
    Model VARCHAR(30),
    Cyclinder INT,
    Price DOUBLE,
    PRIMARY KEY (Car_id)
);
```

2. The application should be able to handle these activities:

- View all data
- Add new data
- Edit/Update current data
- Delete the specific data